

Section 1: The Matrix Foundations (Linear Algebra)

Linear Algebra is the mathematics of high-dimensional space. In AI systems, everything (images, text, audio) is converted into numbers and stored as vectors or matrices. Understanding their operations geometrically is crucial.

Vector

An ordered list of numbers representing a point or direction in space. A vector of size N represents a point in N-dimensional space.

Matrix

A 2D grid of numbers representing a linear transformation. Multiplying a vector by a matrix rotates, scales, or projects that vector in space.

Dot Product

A mathematical operation that takes two equal-length vectors and returns a single scalar. Geometrically, it measures how much two vectors point in the same direction (similarity).

```
import numpy as np
u = np.array([1, 2])
v = np.array([3, 4])
dot_prod = np.dot(u, v) # 1*3 + 2*4 = 11
```

Section 2: The Gradient Currents (Calculus)

Optimization is the core of AI training. Calculus provides the tools to find the parameters that minimize error. The key concepts are derivatives and gradients.

Derivative

The rate at which a function value changes as its input changes. Geometrically, it is the slope of the tangent line to a curve.

Gradient

A vector containing the partial derivatives of a multi-variable function. It points in the direction of steepest ascent of the function. Taking negative steps along the gradient reduces error (Gradient Descent).

Section 3: NumPy Array Manipulation

NumPy provides the fundamental N-dimensional array object (ndarray). It is implemented in C, enabling fast operations via vectorization (avoiding slow Python loops).

```
import numpy as np
# Array creation
a = np.array([[1, 2], [3, 4]])
b = np.ones((2, 2)) # Array of ones

# Element-wise operations
c = a + b # [[2, 3], [4, 5]]

# Transposition
transposed = a.T # [[1, 3], [2, 4]]
```

Broadcasting

A feature in NumPy that allows arithmetic operations between arrays of different shapes by implicitly stretching the smaller array to match the larger one.

Section 4: Pandas DataFrames & Data Cleaning

Pandas is the standard tool for structured table data. Data Sculpting involves cleaning missing values, removing duplicates, and transforming features.

```
import pandas as pd
df = pd.read_csv("data.csv")

# Handling missing values
df['score'] = df['score'].fillna(df['score'].mean())

# Filtering data
filtered_df = df[df['age'] > 21]

# Grouping and Aggregating
summary = df.groupby('category')['sales'].sum()
```

Section 5: Data Visualization

Visualization is how we tell stories with data. Use Matplotlib for simple plotting and Seaborn for statistical visual representations.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df['age'], kde=True)
plt.title("Age Distribution")
# plt.show()
```

Section 6: Checkpoints

Checkpoint 1: Array Arithmetic

Write a Python script that creates a 3x3 matrix representing scores. Multiply it by a vector containing weights. Return the resulting vector and its transpose.

Checkpoint 2: Data Imputation

Take a DataFrame containing employee data with columns 'name', 'salary', and 'department'. Replace missing values in 'salary' with the average salary of their respective department, then remove any duplicate names.

Checkpoint 3: Outlier Detection

Write a script that filters out elements from a Pandas series that are more than 3 standard deviations away from the mean (Z-score filter).

Technical Interview Prep

The following 10 questions represent standard, high-probability questions asked by technical interviewers to evaluate your core understanding of this sequence's materials.

Q1: What is vectorization in NumPy, and why is it faster than standard loops?

Answer: Vectorization is the process of performing array operations without writing explicit loops. It is faster because NumPy delegates the operations to highly-optimized, pre-compiled C-code. It utilizes SIMD (Single Instruction, Multiple Data) CPU instructions to perform operations on multiple data points simultaneously, avoiding Python interpreter overhead.

Q2: Explain NumPy array broadcasting rules.

Answer: NumPy compares array shapes element-wise, starting from the trailing dimensions. Two dimensions are compatible when: 1. They are equal, or 2. One of them is 1. If compatible, the dimension of size 1 is stretched (broadcasted) to match the larger size. If they are not compatible, a `ValueError` is raised.

Q3: What is the difference between `loc` and `iloc` in Pandas?

Answer: `loc` is label-based indexing. You pass row/column names. `iloc` is integer-position-based indexing. You pass row/column integer positions (0-indexed).

Q4: How do you handle missing values in Pandas? Compare deletion vs. imputation.

Answer: You can drop rows/columns with missing data using `df.dropna()` (deletion) or fill them using `df.fillna()` (imputation). Deletion is simple but risks losing valuable data. Imputation (using mean, median, mode, or predictive models) preserves rows but may introduce bias if values are not missing at random.

Q5: What is the difference between a copy and a view in Pandas DataFrames?

Answer: A View is a reference to the original DataFrame's memory. Modifying a view modifies the original data (can cause `SettingWithCopyWarning`). A Copy is a duplicate allocated in new memory; modifying it does not affect the original data.

Q6: What is a matrix dot product geometrically, and how does it differ from element-wise multiplication?

Answer: Element-wise multiplication multiplies corresponding items in arrays of identical shape. A matrix dot product (matrix multiplication) represents a composition of linear transformations. Geometrically, it projects vectors from one space to another.

Q7: Why is the gradient important in machine learning optimization?

Answer: The gradient is a vector pointing in the direction of steepest ascent of a loss function. By calculating the gradient and moving in the opposite direction (gradient descent), we can systematically find weights that minimize loss.

Q8: How does a partial derivative differ from a standard derivative?

Answer: A standard derivative calculates rate of change for single-variable functions. A partial derivative calculates the rate of change of a multi-variable function with respect to one variable, while keeping all other variables constant.

Q9: Explain the difference between `join`, `merge`, and `concat` in Pandas.

Answer: `concat` stacks dataframes along an axis (rows or columns). `merge` joins dataframes based on values in common columns (like SQL joins). `join` combines dataframes based on their index labels.

Q10: How do you identify outliers in a dataset visually and statistically?

Answer: Visually, use box plots or scatter plots. Statistically, use the IQR (Interquartile Range) method (outliers are outside $1.5 * IQR$ from quartiles) or Z-score method (values with Z-score > 3 or < -3).